



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования

«Дальневосточный федеральный университет»

(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ (ШКОЛА)

Департамент информационных и компьютерных систем

АРХИТЕКТУРА ПРЕДПРИЯТИЙ И КОРПОРАТИВНЫХ ИНФОРМАЦИОННЫХ СИСТЕМ

Программа и методические указания

к лабораторной работе № 2

«Моделирование бизнес-архитектуры на основе методологии объектно-ориентированного анализа и проектирования систем»

для студентов направления подготовки

09.04.03 «Прикладная информатика»

магистерская программа «Корпоративные информационные системы
(совместно с ПАО Сбербанк)»

Составитель: проф. П.Г. Рагулин

г. Владивосток

2026

ТЕМА – Объектно-ориентированная методика анализа и проектирования архитектур предприятия и ИС.

Лабораторная работа - Моделирование бизнес-архитектуры на основе методологии объектно-ориентированного анализа и проектирования систем.

Цель работы – изучение вопросов теории и отработка навыков построения графических информационных моделей для объекта автоматизации в CASE среде универсального языка моделирования (UML) для разработки компонентов бизнес-архитектуры предприятия: **диаграммы вариантов использования, диаграммы классов, диаграммы деятельности.**

Инструментальная среда моделирования. Для выполнения лабораторной работы можно использовать свободно распространяемое программное обеспечение для построения диаграмм UML, например:

Umbrello UML Modeller (<https://umbrello.kde.org/>),

ArgoUML (<http://argouml.tigris.org>),

WhiteStarUML 5.8.6 (<https://github.com/StevenTCramer/WhiteStarUml/blob/master/staruml/deploy/License.txt/>) или StarUML (<http://staruml.io>),

Enterprise Architect (<https://ruprogi.ru/software/enterprise-architect>),

Visual Paradigm для UML (<https://ruprogi.ru/software/visual-paradigm>)

или другие программные средства с необходимым инструментарием для создания диаграмм UML.

1. Программа работы

Задание 1. Разработка глоссария

Составить глоссарий базовых терминов по теме лабораторной работы.

Указания к выполнению задания 1

При выборе определений использовать нормативные варианты (из стандартов), из учебников, учебных пособий, справочников и др. т. н. надежных источников.

По каждому определению указать ссылку на источник так, чтобы библиографическое описание источника было приведено в списке литературы.

В список источников **не включать** ресурсы из Интернет *типа студенческих работ*, **не включать** «свободные» ресурсы *типа «wikipedia»*.

Задание 2. Разработка диаграммы вариантов использования (прецедентов) *UML* для заданной системы

Разработать диаграмму вариантов использования (Use case diagram, диаграмма прецедентов) на основе правил универсального языка моделирования *UML* для заданной системы:

- определить модели вариантов использования (прецедентов);
- построить диаграмму вариантов использования (прецедентов);
- составить спецификации вариантов использования (прецедентов).

Требование к диаграмме. При построении диаграммы предусмотреть **не менее двух актеров и не менее двух прецедентов** для каждого актера.

Цель построения диаграммы - моделирование проектируемой системы в форме вариантов использования, с которыми взаимодействуют внешние сущности или актеры (actor).

Оформить в отчете раздел по заданию, включив в него описание всех компонентов диаграммы вариантов использования согласно индивидуальному варианту задания.

Указания к выполнению задания 2

При определении моделей вариантов использования (прецедентов) представить описания характеристик актеров и вариантов использования (прецедентов) в соответствии со структурой (примеры - таблица 1 и 2).

Таблица 1 - Выявление актеров

Актер	Краткое описание
Менеджер	Размещает заказы в производстве и следит за их исполнением. При необходимости может вносить оперативные корректировки в описания заказов, снять заказ с производства.
Диспетчер	Планирует загрузку производственных ресурсов (станков и оборудования) в соответствие с очередью заказов, поступающих от менеджеров. Передаёт планы (сменные задания) в цех. Анализирует фактическое исполнение планов. Вносит оперативные корректировки в планы.
Мастер цеха	Назначает исполнителей каждой из работ сменного задания. Следит за фактическим исполнением работ, отчитывается перед диспетчером.

Таблица 2 - Реестр вариантов использования (прецедентов)

Код	Основной актер	Наименование	Краткое описание
M1	Менеджер	Регистрация заказа	Позволяет менеджеру передавать в производство новые заказы.
M2	Менеджер	Изменение заказа	Менеджер может откорректировать информацию о заказе в производстве.
M3	Менеджер	Удаление заказа.	При необходимости снятия заказа с производства менеджер или диспетчер вызывает функцию «Удаление заказа».
M4	Менеджер либо Диспетчер	Запрос о заказе	Используется менеджером либо диспетчером для поиска нужной информации о состоянии заказа в производстве, необходимой для клиента.
D1	Диспетчер	Планирование нового заказа	Диспетчер размещает вновь поступивший от менеджера заказ в план, в «хвост» очереди.
D2	Диспетчер	Коррекция плана	Диспетчер корректирует план при появлении каких-либо нестыковок.
D3	Диспетчер	Планирование срочного заказа	Диспетчер размещает вновь поступивший от менеджера в требуемое время; очередь заказов смещается.
D4	Диспетчер	Выдача сменного задания	Диспетчер формирует сменное задание для мастера цеха.
C1	Мастер цеха	Назначение исполнителей	Мастер цеха назначает исполнителям (цеховому персоналу) работы из сменного задания.
C1	Мастер цеха	Фиксация результатов	Мастер цеха фиксирует результаты выполнения работы цеховым персоналом.

Спецификации варианта использования представить в соответствии со структурой, приведенной ниже, в примере.

Пример спецификации варианта использования приведен для варианта использования «**D1. Планирование нового заказа**»

D1.1. Краткое описание

Диспетчер размещает вновь поступивший от менеджера заказ в план, в «хвост» очереди. Действующие лица этого прецедента – Диспетчер.

D1.2. Поток событий

Прецедент начинается, когда Диспетчер выбирает деятельность “Планировать новый заказ” из «Главной формы» АРМ «Диспетчер».

D1.2.1. Базовый поток – Планирование нового заказа

0) Диспетчер выбирает «Планировать новый заказ».

- 1) Система отображает список новых заказов, подлежащих планированию.
- 2) Диспетчер выбирает из предложенного списка заказ, который он желает запланировать.
- 3) Систем определяет, что статус заказа – «Обычный».
- 4) Система отображает список работ заказа, отсортированных по очередности исполнения с указанием времени исполнения.
- 5) Система отображает список совместимых ресурсов. Ресурс отображается в том случае, если в заказе есть хотя бы одна работа, для которой он допустим.
- 6) Система отображает для каждого из ресурсов линейки планирования, состоящие из свободных и занятых временных интервалов на шкале времени.
- 7) Диспетчер выбирает работу заказа.
- 8) Система ограничивает набор доступных ресурсов, «затеняя» несовместимые.
- 9) Диспетчер находит на шкале одного из доступных ресурсов интервал необходимого размера и размещает (drag and drop) туда работу заказа.
- 10) Система делает соответствующие отметки в базе данных.

П.п. 7–10 повторяются, пока все работы заказа не будут размещены.

Система удаляет заказ из списка вновь поступивших.

D1.2.2. Альтернативный поток – Планирование по частям

Если при выполнении п. 0 основного потока событий Диспетчеру не удалось обнаружить интервал необходимого раздела, то

Диспетчер выбирает «планировать по частям».

Диспетчер находит на шкале одного из доступных ресурсов интервал произвольного размера и размещает (drag and drop) туда работу заказа.

Система разбивает работу на интервалы и размещает её на свободные позиции выбранного ресурса.

Переход к п. 0 основного потока событий.

D1.2.3. Альтернативный поток – Планирование заказа в срок невозможно

Если Диспетчер обнаружил, что он не может запланировать заказ с соблюдением зафиксированного в заказе срока, то

Диспетчер выбирает «отменить планирование».

Система отправляет уведомление Менеджеру «Заказ №... не может быть спланирован с соблюдением оговоренного с заказчиком срока».

D1.3. Специальные требования

Время планирования одного заказа не должно превышать 3 минут.

D1.4. Предусловия

D1.4.1. Регистрация

Перед тем как начинается этот прецедент, Диспетчер зарегистрирован в системе.

D1.5. Постусловия

При успешном окончании прецедента Диспетчер составляет план, гарантирующий исполнение заказа в срок. При неуспешном – Диспетчер делегирует ответственность за соблюдение сроков исполнения заказа Менеджеру (менеджер указал некорректный срок и должен его откорректировать).

D1.6. Точки расширения

Если при выполнении п. «Планирование нового заказа» выясняется, что заказ имеет статус «Срочный», Система переходит к выполнению расширяющего прецедента «D3. Планирование срочного заказа»

Задание 3. Разработка диаграммы классов *UML* для заданной системы

Разработать диаграмму классов (Class diagram) на основе правил универсального языка моделирования *UML* для заданной системы.

Требование к диаграмме. *При построении диаграммы предусмотреть не менее двух классов, с разными типами отношений между классами.*

Цель построения диаграммы – моделирование различных взаимосвязей между отдельными сущностями предметной области, а также представление их внутренней структуры и типов отношений.

Оформить в отчете раздел по заданию, включив в него описание всех компонентов диаграммы классов согласно индивидуальному варианту задания.

Задание 4. Разработка диаграммы деятельности *UML* для заданной системы

Разработать диаграмму деятельности (Activity diagram) на основе правил универсального языка моделирования *UML* для заданной системы.

Требование к диаграмме. *При построении диаграммы предусмотреть не менее пяти состояний деятельности, с ветвлениями.*

Цель построения диаграммы - моделирование процессов, описывающих взаимодействие объектов.

Оформить в отчете раздел по заданию, включив в него описание всех компонентов диаграммы деятельности согласно индивидуальному варианту задания.

2. Исходные данные

Выбор моделируемой предметно-функциональной деятельности (задачи) проводится в соответствии с выбором предметной экономической деятельности, например вида:

- бухгалтерский учет;
- маркетинг;
- административное управление;
- логистика;
- банковская деятельность;
- страхование;

- транспортировка, складской учет;
- другое (по согласованию).

3. Отчетность

На защиту представить:

файл – «Отчет о работе», - файл в текстовом формате **.docx**, имеющий стандартизованное имя **<Фамилия#ЛР#2.docx>**, где в имени файла текст «**Фамилия**» замещается на соответствующее представление фамилии студента;

файлы диаграмм UML, полученные в выбранном программном (CASE) средстве построения диаграмм UML (в основном формате средства, а также в стандартных форматах сопряжения с другими программными продуктами, при наличии).

Структура отчета

Отчет составляется в соответствии с правилами оформления письменных работ в ДВФУ (указания и рекомендации - см. ниже, **Требования к оформлению отчета**), имеет следующие **структурные компоненты**.

- **Титульный лист** (на отдельном листе, *по форме*);
- **Тема и цель работы**;
- **Бизнес-процессы моделируемой деятельности предприятия** (наименование);
- **Термины и определения** (формируется краткий глоссарий базовых терминов по теме работы, с обязательным указанием источников приводимого определения, из списка литературы – стандарты, учебники, учебные пособия);
- **Описание основной части выполненной работы** (структурируется по заголовкам соответствии с заданиями);
- **Заключение** (кратко указать результаты по выполненным заданиям для заданной системы; указать освоенные навыки при выполнении работы).
- **Литература** (включает библиографическое описание источников по ГОСТ, которые были использованы и на которые в работе сделаны ссылки, в том числе нормативно-правовые материалы – стандарты, ресурсы Интернет).

- **Приложения** (при наличии), содержат *дополнительные материалы* к изложению основной части отчета – листинги программ ЭВМ, таблицы, рисунки, расчеты и т. п.

Требования к оформлению отчета

Отчеты по лабораторным работам относятся к категории «письменная работа», составляются по правилам оформления письменных работ студентами ДВФУ (см. пояснения в лабораторной работе № 1).

4. Методические указания

4.1. Унифицированный язык моделирования UML бизнес-процессов и систем

UML (англ. Unified Modeling Language — унифицированный язык моделирования) — язык графического описания для объектного моделирования бизнес-процессов, системного проектирования, моделирования в области разработки программного обеспечения и отображения организационных структур [1, ..., 13].

UML — это стандартная нотация визуального моделирования программных систем, принятая консорциумом Object Managing Group (OMG) в 1997 г., и на сегодняшний день она поддерживается многими объектно-ориентированными CASE-продуктами, стандартами архитектуры предприятия [1, ..., 6].

UML является языком широкого профиля, это — открытый стандарт, использующий графические обозначения для создания абстрактной модели системы, называемой UML-моделью.

Процесс разработки визуальных моделей в нотации языка UML 2.0 обычно заключается в форме последовательной детализации модельных представлений бизнес-процессов и включает в себя построение ряда диаграмм.

Диаграмма в языке UML – это графическое представление множества элементов, наиболее часто изображается как связный граф из вершин (предметов) и дуг (отношений). Диаграммы рисуются для визуализации деятельности компании с разных точек зрения.

Обычно диаграмма дает неполное представление всех составляющих бизнес-процесса. Один и тот же элемент может появляться во всех диаграммах.

Стандарт **UML** предлагает следующий набор диаграмм для моделирования:

диаграммы вариантов использования (Use case diagrams) – для моделирования бизнес-процессов организации и требований к создаваемой системе;

диаграммы классов (Class diagrams) – для моделирования статической структуры классов системы и связей между ними;

диаграммы поведения системы (Behavior diagrams):

- **диаграммы взаимодействия** (Interaction diagrams):
 - **диаграммы последовательности** (Sequence diagrams) и **кооперативные диаграммы** (Collaboration diagrams) – для моделирования процесса обмена сообщениями между объектами;

диаграммы состояний (Statechart diagrams) – для моделирования поведения объектов системы при переходе из одного состояния в другое;

диаграммы деятельности (Activity diagrams) – для моделирования поведения системы в рамках различных вариантов использования, или моделирования деятельности;

диаграммы реализации (Implementation diagrams):

- **диаграммы компонентов** (Component diagrams) – для моделирования иерархии компонентов (подсистем) системы;
- **диаграммы развертывания** (Deployment diagrams) – для моделирования физической архитектуры системы.

Каждый вид диаграмм имеет характерные черты и рассматривает деятельность компании, ее бизнес-процессы с различных точек зрения.

Для диаграмм прецедентов основной акцент в рассмотрении бизнес-процессов делается на бизнес-задачах, порядке их решения и точках принятия решений, а также на исполнителях и участниках бизнес-задач.

Диаграммы классов позволяют моделировать информационные потоки для бизнес-задач, определяя входные и выходные данные по каждой задаче (операции).

Диаграммы взаимодействия регистрируют последовательность действий в бизнес-процессе с учетом временной оси и взаимоотношений между

участвующими в процессе объектами (в их числе не только исполнители, но различные сервисы, компьютерные системы и другие интерфейсы взаимодействия).

Диаграммы деятельности позволяют составить блок-схему алгоритма реализации конкретного бизнес-процесса с выделением зон ответственности участников процесса за выполнение конкретных операций.

4.2. Диаграммы вариантов использования

Диаграммы вариантов использования описывают функциональное назначение системы или то, что система должна делать в процессе своего функционирования.

Диаграмма вариантов использования (Use case diagram, диаграмма прецедентов) — диаграмма, на которой отражены отношения, существующие между актёрами и вариантами использования.

Создание диаграммы вариантов использования имеет следующие цели:

- определить общие границы и контекст моделируемой предметной области на начальных этапах проектирования системы;
- сформулировать общие требования к функциональному поведению системы;
- разработать исходную концептуальную модель системы для ее последующей детализации в форме логических и физических моделей;
- подготовить исходную документацию для взаимодействия разработчиков системы с ее заказчиками и пользователями.

Диаграмма вариантов использования представляет собой граф специального вида, который является графической нотацией для представления конкретных вариантов использования, актеров и отношений между этими элементами.

Базовыми элементами диаграммы вариантов использования являются вариант использования и актер (actor).

Вариант использования (Use case) — внешняя спецификация последовательности действий, которые система или другая сущность могут выполнять в процессе взаимодействия с актерами.

Вариант использования представляет собой спецификацию общих особенностей поведения или функционирования моделируемой системы без рассмотрения внутренней структуры этой системы.

Каждый вариант использования определяет последовательность действий, которые должны быть выполнены проектируемой системой при взаимодействии ее с соответствующим актером, сами эти действия не изображаются на рассматриваемой диаграмме.

Содержание варианта использования может быть представлено в форме дополнительного пояснительного текста, который раскрывает смысл или семантику действий при выполнении данного варианта использования. Такой пояснительный текст получил название текста-сценария или просто сценария.

Вариант использования обозначается на диаграмме эллипсом, внутри которого содержится его имя в форме существительного (рисунок 1, а) или глагола (рисунок 1, б) с пояснительными словами. Сам текст имени должен начинаться с заглавной буквы.

Имя - строка текста, которая используется для идентификации любого элемента модели.

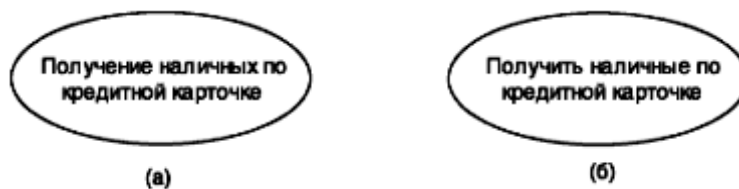


Рисунок 1 - Графическое обозначение варианта использования

Каждый вариант использования соответствует отдельному сервису, который предоставляет моделируемая система по запросу актера, т. е. определяет один из способов применения системы.

Сервис, который инициализируется по запросу актера, должен представлять собой законченную последовательность действий (после того как система закончит обработку запроса актера, она должна возвратиться в исходное состояние, в котором снова готова к выполнению следующих запросов).

Диаграмма вариантов использования содержит конечное множество вариантов использования, которые в целом должны определять все возможные стороны ожидаемого поведения системы.

Для удобства множество вариантов использования может рассматриваться как отдельный пакет.

Актер (actor) — согласованное множество ролей, которые играют внешние сущности по отношению к вариантам использования при взаимодействии с ними.

Актер представляет собой любую внешнюю по отношению к моделируемой системе сущность, которая взаимодействует с системой и использует ее функциональные возможности для достижения определенных целей или решения задач.

Каждый актер может рассматриваться как некая отдельная роль относительно конкретного варианта использования.

Стандартным графическим обозначением актера на диаграммах является фигурка «человечка», под которой записывается имя актера (рисунок 2).

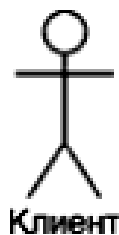


Рисунок 2 - Графическое обозначение актера

В некоторых случаях актер может обозначаться в виде прямоугольника класса со стереотипом $\langle \rangle$ и обычными составляющими элементами класса.

Имена актеров должны начинаться с заглавной буквы и следовать рекомендациям использования имен для типов и классов модели. При этом символ отдельного актера связывает соответствующее описание актера с конкретным именем.

Актеры используются для моделирования внешних по отношению к проектируемой системе сущностей, которые взаимодействуют с системой.

В качестве актеров могут выступать другие системы, в том числе подсистемы проектируемой системы или ее отдельные классы. Наиболее

наглядный пример актера — конкретный посетитель web-сайта в Интернет со своими параметрами аутентификации.

Актеры взаимодействуют с системой посредством передачи и приема сообщений от вариантов использования.

Сообщение представляет собой запрос актером сервиса от системы и получение этого сервиса. Это взаимодействие может быть выражено посредством ассоциаций между отдельными актерами и вариантами использования.

Кроме этого, с актерами могут быть связаны интерфейсы, которые определяют, каким образом другие элементы модели взаимодействуют с этими актерами (например, датчик, устройство считывания штрих-кода и т.д.).

Интерфейс обозначается в виде маленького круга, рядом с которым записывается его имя. В качестве имени может быть существительное или строка текста. Если имя записывается на английском языке, то оно начинается с заглавной буквы I.

Графический символ интерфейса соединяется с тем вариантом использования, который его поддерживает, на диаграмме двумя способами:

- сплошной линией (указывает, что связанный с интерфейсом вариант использования должен реализовывать все необходимые для него сервисы);
- пунктирной линией со стрелкой (означает, что вариант использования предназначен для спецификации только того сервиса, который необходим для реализации данного интерфейса).

Примечание (Note) предназначено для включения в модель произвольной текстовой информации, имеющей непосредственное отношение к контексту разрабатываемого проекта (комментарии разработчика - дата; ограничения - на значения отдельных связей или экземпляры сущностей; помеченные значения).

Графически обозначаются прямоугольником с «загнутым» верхним правым углом (рисунок 3). Примечание может относиться к любому элементу диаграммы, в этом случае их соединяет пунктирная линия. Если примечание относится к нескольким элементам, то от него проводятся несколько линий.

Отношение (Relationship) — семантическая связь между отдельными элементами модели.

Между элементами диаграммы вариантов использования могут существовать различные отношения, которые описывают взаимодействие экземпляров одних актеров и вариантов использования с экземплярами других актеров и вариантов.

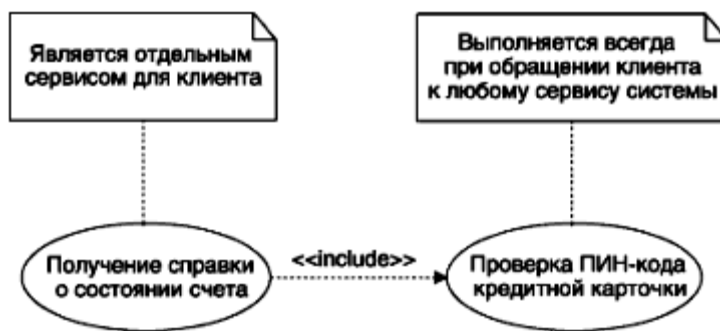


Рисунок 3 - Примеры примечаний на диаграммах вариантов использования

Один актер может взаимодействовать с несколькими вариантами использования.

В свою очередь один вариант использования может взаимодействовать с несколькими актерами, предоставляя для всех них свой сервис.

В языке UML имеется несколько стандартных **видов отношений между актерами и вариантами использования**:

- ассоциации (Association relationship);
- включения (Include relationship);
- расширения (Extend relationship);
- обобщения (Generalization relationship).

Общие свойства вариантов использования могут быть представлены тремя различными способами: с помощью отношений включения, расширения и обобщения.

Отношение ассоциации используется при построении всех графических моделей систем.

Применительно к диаграммам вариантов использования ассоциация служит для обозначения конкретной роли актера при его взаимодействии с отдельным вариантом использования.

На диаграмме вариантов использования, так же, как и на других диаграммах, отношение ассоциации обозначается сплошной линией между

актером и вариантом использования. Эта линия может иметь некоторые дополнительные обозначения, например имя и кратность (рисунок 4).

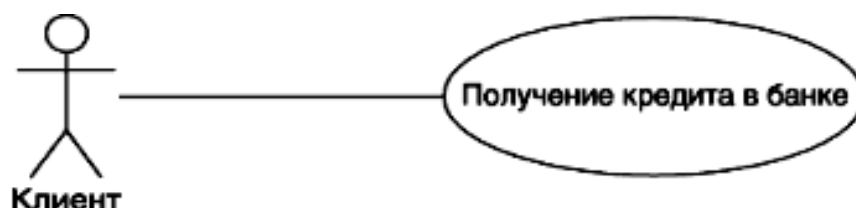


Рисунок 4 - Пример графического представления отношения ассоциации между актером и вариантом использования

Отношение включения (Include) — это разновидность отношения зависимости между базовым вариантом использования и его специальным случаем.

Отношение включения устанавливается между двумя вариантами использования и указывает на то, что заданное поведение для одного варианта использования включается в качестве составного фрагмента в последовательность поведения другого варианта использования.

Графически отношение включения обозначается в форме пунктирной линии со стрелкой, направленной от базового варианта использования к включаемому варианту использования. При этом данная линия помечается стереотипом `<<include>>`, как показано на рисунок 5.



Рисунок 5 - Пример графического изображения отношения включения между вариантами использования

Отношение включения, направленное от варианта использования «Предоставление кредита в банке» к варианту использования «Проверка платежеспособности клиента», указывает на то, что каждый экземпляр первого варианта использования всегда включает в себя функциональное поведение или выполнение второго варианта использования (поведение второго

варианта использования является частью поведения первого варианта использования на данной диаграмме).

Один вариант использования может входить в несколько других вариантов, а также содержать в себе другие варианты. Включаемый вариант использования является независимым от базового варианта в том смысле, что он предоставляет последнему инкапсулированное поведение, детали реализации которого скрыты от последнего и могут быть легко перераспределены между несколькими включаемыми вариантами использования. Базовый вариант зависит только от результатов выполнения включаемого в него варианта использования, но не от структуры включаемых в него вариантов.

Отношение расширения (Extend) определяет взаимосвязь базового варианта использования с другим вариантом использования, функциональное поведение которого задействуется базовым не всегда, а только при выполнении дополнительных условий.

Графически обозначается в форме пунктирной линии со стрелкой, направленной от того варианта использования, который является расширением для базового варианта использования. Данная линия со стрелкой должна быть помечена стереотипом `<<extend>>`, как показано на рисунк 6.



Рисунок 6 - Пример графического изображения отношения расширения между вариантами использования

В изображенном фрагменте имеет место отношение расширения между базовым вариантом использования «Предоставление кредита в банке» и вариантом использования «Предоставление налоговых льгот». Это означает, что свойства поведения первого варианта использования в некоторых случаях могут быть дополнены функциональностью второго варианта использования. Для того чтобы это расширение имело место, должно быть выполнено определенное логическое условие данного отношения расширения.

Если базовый вариант использования выполняет некоторую последовательность действий, которая определяет его поведение, и при этом имеется точка расширения на экземпляр другого варианта использования, то

проверяется логическое условие данного отношения. Если это условие выполняется, исходная последовательность действий расширяется посредством включения действий другого варианта использования. Условие отношения расширения проверяется один раз — при первой ссылке на точку расширения, и если оно выполняется, то все расширяющие варианты использования вставляются в базовый вариант.

Отношение обобщения используется тогда, когда два и более актера могут иметь общие свойства, т. е. взаимодействовать с одним и тем же множеством вариантов использования одинаковым образом. Такая общность свойств и поведения представляется в виде отношения обобщения с другим актером, который моделирует соответствующую общность ролей.

Графически обозначается сплошной линией со стрелкой в форме не закрашенного треугольника, которая указывает на родительский вариант использования (рисунок 7). Эта линия со стрелкой имеет специальное название — стрелка-обобщение.

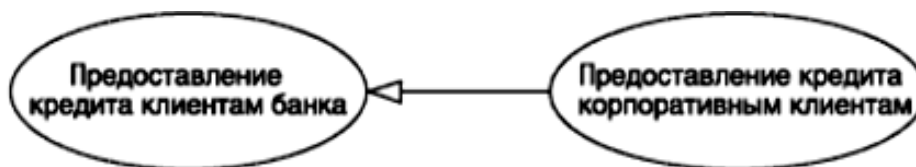


Рисунок 7 - Пример графического изображения отношения обобщения между вариантами использования

В данном примере отношение обобщения указывает на то, что вариант использования «Предоставление кредита корпоративным клиентам» - специальный случай варианта использования «Предоставление кредита клиентам банка». Другими словами, первый вариант использования является специализацией второго варианта использования.

Вариант использования «Предоставление кредита клиентам банка» еще называют предком или родителем по отношению к варианту использования «Предоставление кредита корпоративным клиентам», а последний - потомком по отношению к первому варианту использования.

Потомок наследует все свойства поведения своего родителя, а также может обладать дополнительными особенностями поведения.

Отношение обобщения между вариантами использования применяется в том случае, когда необходимо отметить, что дочерние варианты

использования обладают всеми особенностями поведения родительских вариантов и участвуют во всех отношениях родительских вариантов.

Язык UML включает в себя еще и дополнительные графические обозначения, ориентированные для решения задач из определенной предметной области. Примеры подобных обозначений, которые используются для моделирования бизнес-систем и могут быть изображены на диаграммах вариантов использования: бизнес-актер, сотрудник и бизнес-вариант использования.

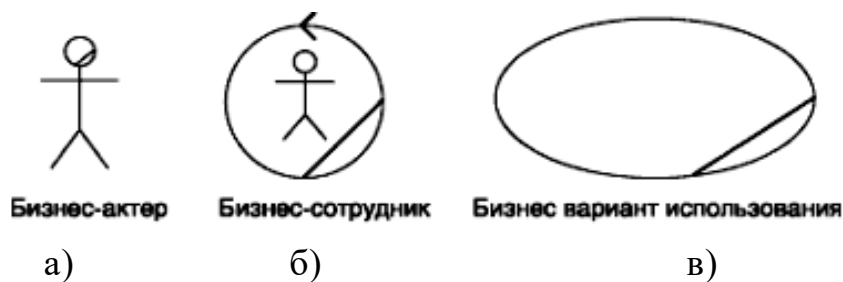


Рисунок 8 - Графические изображения бизнес-актера (а), бизнес-сотрудника (б) и бизнес-варианта использования (в)

Применение этих элементов графической нотации иллюстрирует пример представления диаграмм вариантов использования для системы продажи товаров в супермаркете.

Одно из главных назначений диаграммы вариантов использования заключается в формализации функциональных требований к системе.

Общее количество актеров в модели не должно превышать 20, а вариантов использования - 50, в противном случае модель теряет свою наглядность.

Для разработки диаграммы вариантов использования рекомендуется некоторая последовательность действий:

- определить главных и второстепенных актеров;
- определить цели главных актеров по отношению к системе;
- сформулировать основные варианты использования, которые специфицируют функциональные требования к системе;
- упорядочить варианты использования по степени убывания риска их реализации;
- рассмотреть все базовые варианты использования в порядке убывания их степени риска;

- выделить участников, интересы, условия выполнения выбранного варианта использования;
- написать сценарий реализации выбранного варианта использования;
- определить исключения в выполнении сценария варианта использования;
- написать сценарии для всех исключений;
- выделить общие варианты использования и изобразить их взаимосвязи с базовыми со стереотипом $\langle \rangle$;
- выделить варианты использования для исключений и изобразить их взаимосвязи с базовыми со стереотипом $\langle \rangle$;
- проверить диаграмму на отсутствие дублирования вариантов использования и актеров.

Пример. Информационная система включает базу данных «Автоматизация процесса составления расписания в учебном заведении» (рисунок 9).

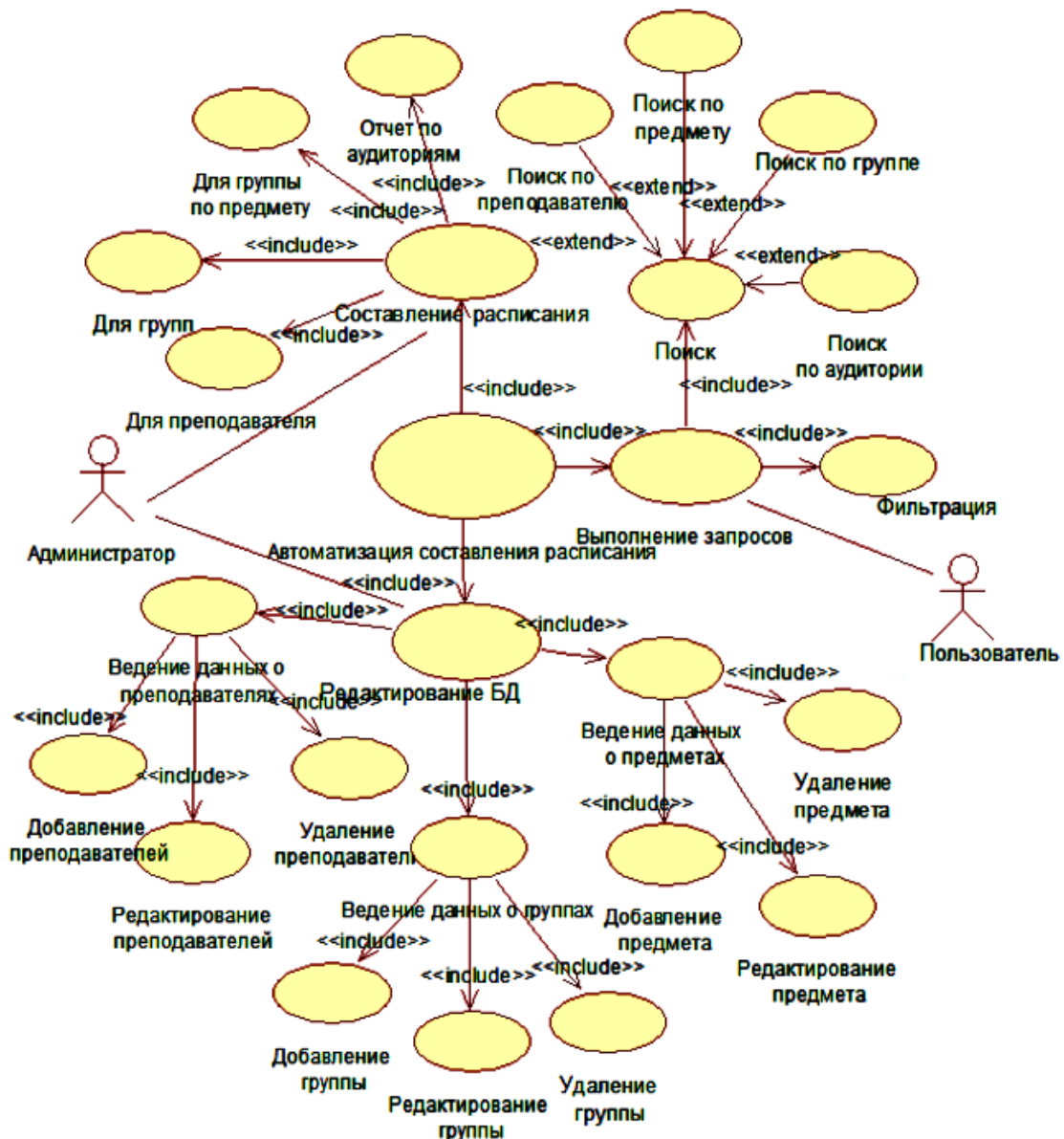


Рисунок 9 - Пример диаграммы вариантов использования

Система обеспечивает корректировку данных, а именно в БД «Группы» может изменяться перечень предметов в соответствии с курсом группы и отделением; в БД «Предметы» могут изменяться номера аудиторий и фамилии преподавателей; осуществляет поиск по ФИО преподавателя, номеру аудитории, названию предмета и номеру группы.

Система составляет расписание работы для конкретного преподавателя на неделю; для группы на неделю; для группы по конкретному предмету, а также отчет о загрузке аудиторий на каждый день.

4.3. Диаграмма классов

Диаграмма классов (Class diagram) — статическая структурная диаграмма, описывающая структуру системы, демонстрирующая классы системы, их атрибуты, методы и зависимости между классами.

Существуют разные точки зрения на построение диаграмм классов в зависимости от целей их применения:

- концептуальная точка зрения — диаграмма классов описывает модель предметной области, в ней присутствуют только классы прикладных объектов;
- точка зрения спецификации — диаграмма классов применяется при проектировании информационных систем;
- точка зрения реализации — диаграмма классов содержит классы, используемые непосредственно в программном коде (при использовании объектно-ориентированных языков программирования).

Диаграмма классов отражает различные взаимосвязи между отдельными сущностями предметной области, а также описывает их внутреннюю структуру и типы отношений.

Диаграмма классов (Class diagram) — диаграмма языка UML, на которой представлена совокупность декларативных или статических элементов модели, таких как классы с атрибутами и операциями, а также связывающие их отношения.

Диаграмма классов предназначена для представления статической структуры модели системы в терминологии классов объектно-ориентированного программирования.

Диаграмма классов содержит интерфейсы, пакеты, отношения и даже отдельные экземпляры классификаторов, такие как объекты и связи.

Класс (Class) — абстрактное описание множества однородных объектов, имеющих одинаковые атрибуты, операции и отношения с объектами других классов.

Графически изображается в виде прямоугольника, который дополнительно может быть разделен горизонтальными линиями на разделы или секции (рисунок 10). В этих секциях могут указываться имя класса, атрибуты и операции класса. Даже если секции атрибутов и операций пусты, в обозначении класса они должны быть выделены горизонтальной линией.

В первом случае для класса «Окружность» (рисунок 11, а) указаны только его атрибуты – точка на координатной плоскости, которая определяет расположение ее центра. Для класса «Окно» (рисунок 11, б) указаны только его операции, при этом секция его атрибутов оставлена пустой. Для класса «Счет» (рисунок 11, в) дополнительно изображена четвертая секция, в которой указано требование – реализовать резервное копирование объектов этого класса.



Рисунок 10 - Варианты графического изображения класса на диаграмме классов



Рисунок 11 - Примеры графического изображения конкретных классов

Имя класса должно быть уникальным в пределах пакета, который может содержать одну или несколько диаграмм классов. Имя класса записывается по центру секции имени полужирным шрифтом и должно начинаться с заглавной буквы.

В секции имени класса могут также находиться стереотипы или ссылки на стандартные шаблоны, от которых образован данный класс и, соответственно, от которых он наследует атрибуты и операции. Здесь также могут записываться и другие общие свойства этого класса.

Класс может иметь или не иметь экземпляров или объектов. В зависимости от этого в языке UML различают конкретные и абстрактные классы.

Конкретный класс (Concrete class) — класс, на основе которого могут быть непосредственно созданы экземпляры или объекты.

Рассмотренные выше обозначения относятся к конкретным классам.

Абстрактный класс (Abstract class) — класс, который не имеет экземпляров или объектов.

Для обозначения имени абстрактного класса используется курсив.

Когда необходимо явно указать, к какому пакету относится тот или иной класс, используется специальный символ разделитель – двойное двоеточие - (::). Синтаксис строки имени класса в этом случае будет следующий:

<Имя пакета>::**Имя класса**

Например, если определен пакет с именем Банк, то класс Счет в этом банке может быть записан в виде: Банк::Счет.

Атрибут (Attribute) — содержательная характеристика класса, описывающая множество значений, которые могут принимать отдельные объекты этого класса.

Атрибут класса служит для представления отдельного свойства или признака, который является общим для всех объектов данного класса. Каждому атрибуту класса соответствует отдельная строка текста, которая состоит из квантора видимости атрибута, имени атрибута, его кратности, типа значений атрибута и, возможно, его исходного значения.

Общий формат записи отдельного атрибута класса, следующий:

<квантор видимости> <имя атрибута> [кратность] : <тип атрибута> = <исходное значение> {строка-свойство}

Видимость (Visibility) — качественная характеристика описания элементов класса, характеризующая потенциальную возможность других объектов модели оказывать влияние на отдельные аспекты поведения данного класса.

Видимость в языке UML специфицируется с помощью *квантора видимости* (может быть опущен), который может принимать одно из 4-х возможных значений и отображаться при помощи специальных символов.

- "+" – область видимости типа общедоступный (public). Атрибут с этой областью видимости доступен или виден из любого другого класса пакета, в котором определена диаграмма.

- "#" – область видимости типа защищенный (protected). Атрибут недоступен или невиден для всех классов, за исключением подклассов данного класса.
- "-" – область видимости типа закрытый (private). Атрибут недоступен или невиден для всех классов без исключения.
- "" – область видимости типа пакетный (package). Атрибут недоступен или невиден для всех классов за пределами пакета, в котором определен класс-владелец данного атрибута.

Вместо условных графических обозначений можно записывать соответствующее ключевое слово: public, protected, private, package.

Имя атрибута используется в качестве идентификатора соответствующего атрибута и поэтому должно быть уникальным в пределах данного класса. Имя атрибута - обязательный элемент, должно начинаться со строчной (малой) буквы и не должно содержать пробелов.

Кратность (multiplicity) — спецификация области значений допустимой мощности, которой могут обладать соответствующие множества.

Кратность атрибута характеризует общее количество конкретных атрибутов данного типа, входящих в состав отдельного класса:

[нижняя граница .. верхняя граница].

Интервалов кратности для отдельного атрибута может быть несколько. Если в качестве кратности указывается единственное число, то кратность атрибута принимается равной данному числу. Если же указывается «*», то кратность атрибута может быть произвольным положительным целым числом или нулем.

Тип атрибута представляет собой выражение, семантика которого определяется некоторым типом данных, определенным в пакете Типы данных языка UML или самим разработчиком, или в зависимости от языка программирования, который предполагается использовать для реализации данной модели.

Исходное значение служит для задания начального значения соответствующего атрибута в момент создания отдельного экземпляра класса. Если оно не указано, то значение соответствующего атрибута не определено на момент создания нового экземпляра класса.

При задании атрибутов могут быть использованы дополнительные синтаксические конструкции — это подчеркивание строки атрибута, пояснительный текст в фигурных скобках и косая черта перед именем атрибута. Подчеркивание строки атрибута означает, что соответствующий атрибут общий для всех

объектов данного класса, т. е. его значение у всех создаваемых объектов одинаковое (аналог ключевого слова `static` в некоторых языках программирования).

Пояснительный текст в фигурных скобках может означать две различные конструкции. Если в этой строке имеется знак равенства, то вся запись Строка-свойство служит для указания дополнительных свойств атрибута, которые могут характеризовать особенности изменения значений атрибута в ходе выполнения программы. Фигурные скобки как раз и обозначают фиксированное значение соответствующего атрибута для класса в целом, которое должны принимать все вновь создаваемые экземпляры класса без исключения. Это значение принимается за исходное значение атрибута, которое не может быть переопределено в последующем.

Отсутствие строки-свойства по умолчанию трактуется так, что значение соответствующего атрибута может быть изменено в программе.

Знак «/» перед именем атрибута указывает на то, что данный атрибут является производным от некоторого другого атрибута этого же класса.

Производный атрибут (Derived element) — атрибут класса, значение которого для отдельных объектов может быть вычислено посредством значений других атрибутов этого же объекта.

Операция (Operation) — это сервис, предоставляемый каждым экземпляром или объектом класса по требованию своих клиентов, в качестве которых могут выступать другие объекты, в том числе и экземпляры данного класса.

Общий формат записи отдельной операции класса, следующий:

<квантор видимости> <имя операции> (список параметров):

<выражение типа возвращаемого значения> {строка-свойство}

Квантор видимости операции аналогичен квантору видимости атрибутов.

Имя операции — обязательный элемент, должно начинаться со строчной (малой) буквы, и записываться без пробелов.

Список параметров является перечнем разделенных запятой формальных параметров, каждый из которых, в свою очередь, может быть представлен в следующем виде:

<направление параметра> <имя параметра> : <выражение типа> = <значение параметра по умолчанию>

Параметр (Parameter) — спецификация переменной операции, которая может быть изменена, передана или возвращена.

Параметр может включать имя, тип, направление и значение по умолчанию. Направление параметра — есть одно из ключевых слов in, out или inout со значением in по умолчанию, в случае если вид параметра не указывается. Имя параметра есть идентификатор соответствующего формального параметра, при записи которого следуют правилам задания имен атрибутов.

Выражение типа возвращаемого значения указывает на тип данных значения, которое возвращается объектом после выполнения соответствующей операции. Две точки и выражение типа возвращаемого значения могут быть опущены, если операция не возвращает никакого значения. Для указания нескольких возвращаемых значений данный элемент спецификации операции может быть записан в виде списка отдельных выражений.

Операция с областью действия на весь класс показывается подчеркиванием имени и строки выражения типа. В этом случае под областью действия операции понимаются все объекты этого класса. В этом случае вся строка записи операции подчеркивается.

Строка-свойство служит для указания значений свойств, которые могут быть применены к данной операции. Строка-свойство может отсутствовать, если свойства не специфицированы.

Список формальных параметров и тип возвращаемого значения не обязателен.

В рамках профиля для процесса разработки программного обеспечения (ПО) языка UML (The UML Profile for Software Development Processes) можно рассмотреть три специальных графических примитива, которые могут быть использованы для уточнения семантики отдельных классов при построении различных диаграмм:

Управляющий класс (control class) — класс, отвечающий за координацию действий других классов. На каждой диаграмме классов должен быть хотя бы один управляющий класс, причем количество посылаемых объектам управляющего класса сообщений мало, по сравнению с числом рассылаемых ими.

Управляющий класс отвечает за координацию действий других классов. Как правило, данный класс является активным и инициирует рассылку множества сообщений другим классам модели. Кроме специального обозначения управляющий класс может быть изображен в форме прямоугольника класса со стереотипом $\diamond$ (рисунок 12, а).

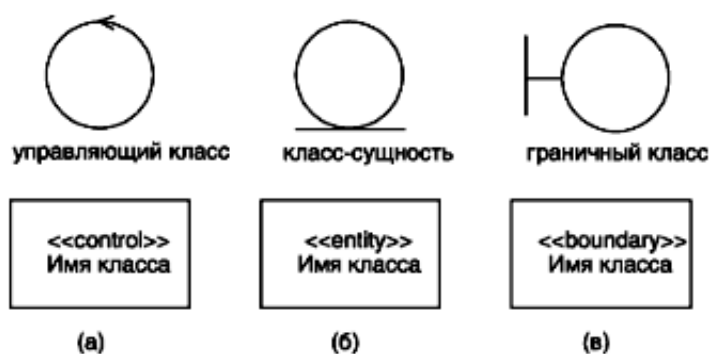


Рисунок 12 - Графическое изображение классов для моделирования ПО

Класс-сущность (Entity class) — пассивный класс, информация о котором должна храниться постоянно и не уничтожаться с выключением системы. Класс-сущность содержит информацию, которая должна храниться постоянно и не уничтожается с уничтожением объектов данного класса или прекращением работы моделируемой системы, связанные с выключением системы или завершением программы.

Как правило, класс-сущность соответствует отдельной таблице базы данных. В этом случае его атрибуты являются полями таблицы, а операции – присоединенными или хранимыми процедурами. Этот класс пассивный и лишь принимает сообщения от других классов модели. Класс-сущность может быть изображен также стандартным образом в форме прямоугольника класса со стереотипом $\diamond$ (рисунок 12, б).

Граничный класс (Boundary class) — класс, который располагается на границе системы с внешней средой и непосредственно взаимодействует с актерами, но является составной частью системы. Граничный класс может быть изображен также стандартным образом в форме прямоугольника класса со стереотипом $\diamond$ (рисунок 12, в).

В рамках профиля для бизнес-моделирования языка UML (The UML Profile for Business Modeling) можно рассмотреть три специальных графических примитива, которые могут быть использованы для уточнения семантики отдельных классов при построении моделей бизнес-систем:

Сотрудник (Business worker) — класс, служащий на диаграмме классов для представления любого сотрудника, который является элементом бизнес-системы и взаимодействует с другими сотрудниками при реализации бизнес-процесса. Этот класс также может быть изображен в форме прямоугольника класса со стереотипом $\diamond$ или $\diamond$ (рисунок 13, а).

Сотрудник для связи с окружением (caseworker) – класс, служащий для представления в бизнес-системе такого сотрудника, который, являясь элементом бизнес-системы, непосредственно взаимодействует с актерами (бизнес-актерами) при реализации бизнес-процесса. Этот класс также может быть изображен в форме прямоугольника класса со стереотипом $\diamond$ (рисунок 13, б).

Бизнес-сущность (Business entity) — специальный случай класса-сущности, который также не инициирует никаких сообщений. Этот класс служит для сохранения информации о результатах выполнения бизнес-процесса в моделируемой бизнес-системе или организации. Этот класс также может быть изображен в форме прямоугольника класса со стереотипом $\diamond$ (рисунок 13, в).

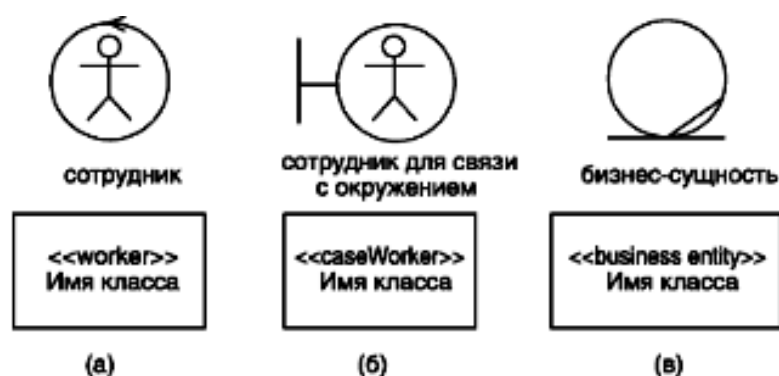


Рисунок 13 - Графическое изображение классов для моделирования бизнес-систем

Кроме внутреннего устройства классов важную роль при разработке проектируемой системы имеют различные отношения между классами, которые также могут быть изображены на диаграмме классов. Базовые отношения, изображаемые на диаграммах классов:

- отношение ассоциации (Association relationship);
- отношение обобщения (Generalization relationship);
- отношение агрегации (Aggregation relationship);
- отношение композиции (Composition relationship).

Отношение ассоциации соответствует наличию произвольного отношения или взаимосвязи между классами. Оно обозначается сплошной линией со стрелкой или без нее и с дополнительными символами, которые характеризуют специальные свойства ассоциации.

Ассоциации применительно к диаграммам классов немного шире, чем применительно к диаграммам вариантов использования. В качестве дополнительных

специальных символов могут использоваться имя ассоциации, символ навигации, а также имена и кратность классов-ролей ассоциации. Имя ассоциации - обязательный элемент ее обозначения. Однако если оно задано, то записывается с заглавной буквы рядом с линией ассоциации.

Отдельные классы ассоциации играют определенную роль в соответствующем отношении, на что явно указывает имя конечных точек ассоциации на диаграмме.

Наиболее простой случай данного отношения - *бинарная ассоциация*, которая служит для представления произвольного отношения между двумя классами. Она связывает в точности два различных класса и может быть ненаправленным (симметричным) или направленным отношением.

Частный случай бинарной ассоциации – *рефлексивная ассоциация*, которая связывает класс с самим собой. Ненаправленная бинарная ассоциация изображается линией без стрелки. Для нее на диаграмме может быть указан порядок чтения классов с использованием значка в форме треугольника рядом с именем данной ассоциации.

В качестве простого примера ненаправленной бинарной ассоциации можно рассмотреть отношение между двумя классами - классом «Компания» и классом «Сотрудник» (рисунок 14). Они связаны между собой бинарной ассоциацией «Работает». Для данного отношения определен следующий порядок чтения следования классов - сотрудник работает в компании.

Направленная бинарная ассоциация изображается сплошной линией с простой стрелкой на одной из ее конечных точек. Направление этой стрелки указывает на то, какой класс является первым, а какой - вторым.

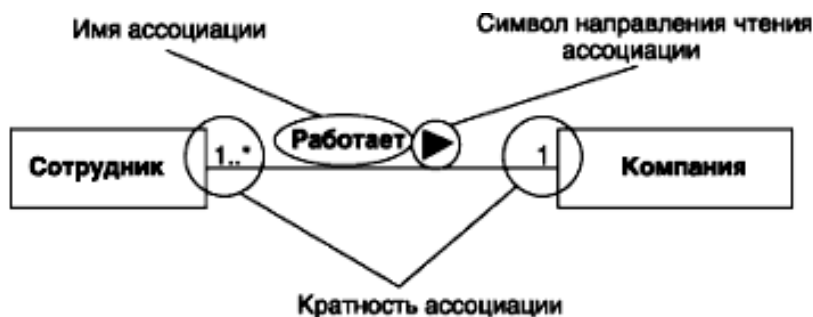


Рисунок 14 - Графическое изображение ненаправленной бинарной ассоциации между классами

В качестве простого примера направленной бинарной ассоциации можно рассмотреть отношение между двумя классами - классом «Клиент» и классом «Счет» (рисунок 15). Они связаны между собой бинарной ассоциацией с именем «Имеет», для которой определен порядок следования классов. Это означает, что конкретный объект класса «Клиент» всегда должен указываться первым при рассмотрении взаимосвязи с объектом класса «Счет», например, <клиент, счет_1, счет_2, ..., счет_n>.

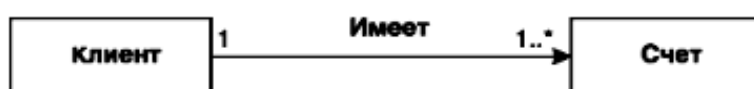


Рисунок 15 - Графическое изображение направленной бинарной ассоциации между классами

Частный случай отношения ассоциации - так называемая *исключающая ассоциация* (Xor-association). Семантика данной ассоциации указывает на то, что из нескольких потенциально возможных вариантов данной ассоциации в каждый момент времени может использоваться только один.

На диаграмме классов исключающая ассоциация изображается пунктирной линией, соединяющей две и более ассоциации (рисунок 16), рядом с которой записывается ограничение в форме строки текста в фигурных скобках: {xor}.

Тернарная ассоциация связывает отношением три класса. Ассоциация более высокой арности называется n-арной ассоциацией.

n-арная ассоциация - ассоциация между тремя и большим числом классов.

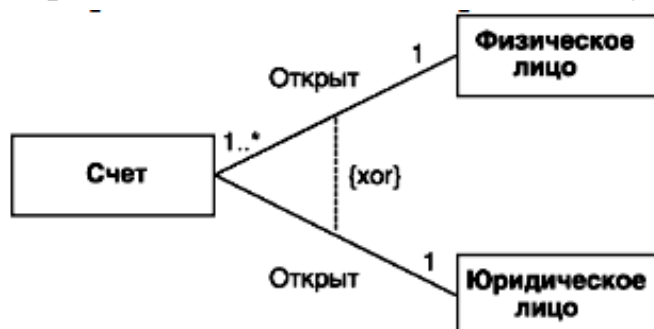


Рисунок 16 - Графическое изображение исключающей ассоциации между тремя классами

Каждый экземпляр такой ассоциации представляет собой упорядоченный набор (кортеж), содержащий n экземпляров из соответствующих классов. Такая

ассоциация связывает отношением более чем три класса, при этом класс может участвовать в ассоциации более чем один раз.

Графически n-арная ассоциация обозначается ромбом, от которого ведут линии к символам классов данной ассоциации. Имя n-арной ассоциации записывается рядом с ромбом соответствующей ассоциации. Однако порядок классов в n-арной ассоциации, в отличие от порядка множеств в отношении, на диаграмме не фиксируется.

В качестве примера тернарной ассоциации можно рассмотреть отношение между тремя классами: «Сотрудник», «Компания» и «Проект». Данная ассоциация указывает на наличие отношения между этими тремя классами, которое может представлять информацию о проектах, реализуемых в компании, и о сотрудниках, участвующих в выполнении отдельных проектов (рисунок 17).



Рисунок 17 - Графическое изображение тернарной ассоциации между тремя классами

Класс может быть присоединен к линии ассоциации пунктирной линией, т. е. обеспечивать поддержку свойств соответствующей n-арной ассоциации, а сама n-арная ассоциация имеет атрибуты, операции и/или ассоциации. Такой класс является ассоциативным классом.

Класс ассоциации (Association class) - модельный элемент, который одновременно является ассоциацией и классом.

Отдельный класс в ассоциации может играть определенную роль в данной ассоциации. Поэтому вводится специальный элемент - *концевая точка ассоциации*, которая графически соответствует точке соединения линии ассоциации с отдельным классом.

Конец ассоциации (Association end) - концевая точка ассоциации, которая связывает ассоциацию с классификатором. Конец ассоциации - часть

ассоциации, но не класса. Каждая ассоциация может иметь два или больше концов ассоциации.

Роль (Role) - имеющее имя специфическое поведение некоторой сущности, рассматриваемой в определенном контексте. Роль может быть статической или динамической.

Имя роли представляет собой строку текста рядом с концом ассоциации для соответствующего класса. Она указывает на специфическую роль, которую играет класс, являющийся концом рассматриваемой ассоциации. Имя роли не обязательный элемент обозначений.

Кратность ассоциации относится к концам ассоциации и обозначается в виде интервала целых чисел, аналогично кратности атрибутов и операций классов, но без прямых скобок. Этот интервал записывается рядом с концом соответствующей ассоциации и означает потенциальное число отдельных экземпляров класса, которые могут иметь место, когда остальные экземпляры или объекты классов фиксированы.

Так, для примера (рис. 18) кратность " 1 " для класса «Компания» означает, что каждый сотрудник может работать только в одной компании. Кратность " 1..* " для класса «Сотрудник» означает, что в каждой компании могут работать несколько сотрудников, общее число которых заранее неизвестно и ничем не ограничено. Вместо кратности " 1..* " нельзя записать только символ " * ", поскольку последний означает кратность " 0..* ". Для данного примера это означало бы, что отдельные компании могут совсем не иметь сотрудников в своем штате. Такая кратность приемлема в ситуациях, когда в компании вообще не выполняется никаких проектов.

Имя ассоциации рассматривается в качестве отдельного атрибута у соответствующих классов ассоциаций и может быть указано на диаграмме явным способом в определенной секции прямоугольника класса.

Ассоциация является наиболее общей формой отношения в языке UML. Все другие типы рассматриваемых отношений можно считать частным случаем данного отношения.

Отношение обобщения является отношением классификации между более общим элементом (родителем или предком) и более частным или специальным элементом (дочерним или потомком).

Менее общий элемент модели должен быть согласован с более общим элементом и может содержать дополнительную информацию. Отношение

обобщения описывает иерархическое строение классов и наследование их свойств и поведения.

Согласно одному из главных принципов методологии ООАП - наследованию, класс-потомок обладает всеми свойствами и поведением класса-предка, а также имеет собственные свойства и поведение, которые могут отсутствовать у класса-предка.

Отношение обобщения обозначается сплошной линией с треугольной стрелкой на одном из концов (рисунок 18). Стрелка указывает на более общий класс (класс-предок или суперкласс), а ее начало - на более специальный класс (класс-потомок или подкласс).



Рисунок 18 - Графическое изображение отношения обобщения в языке UML

От одного класса-предка одновременно могут наследовать несколько классов-потомков. т.е. в класс-предок входит несколько линий со стрелками.

Пример: класс «Транспортное средство» (курсив обозначает абстрактный класс) может выступать в качестве суперкласса для подклассов, соответствующих конкретным транспортным средствам, таким как: «Автомобиль», «Автобус», «Трактор» и другим (рисунок 19).



Рисунок 19 - Пример графического изображения отношения:

- а) обобщения для нескольких классов-потомков;
- б) альтернативный вариант обобщения классов для случая объединения отдельных линий

Графическое изображение отношения обобщения по форме соответствует иерархическому дереву, где класс-предок является корнем дерева, а классы-потомки - его листьями.

В дополнение к простой стрелке обобщения может быть присоединена строка текста, указывающая на специальные свойства этого отношения в форме ограничения. Этот текст будет относиться ко всем линиям обобщения, которые идут к классам-потомкам. Поскольку отмеченное свойство касается всех подклассов данного отношения, именно поэтому спецификация этого свойства осуществляется в форме ограничения, которое должно быть записано в фигурных скобках.

В качестве ограничений могут быть использованы следующие ключевые слова языка UML:

- {Complete} - означает, что в данном отношении обобщения специфицированы все классы-потомки, и других классов-потомков у данного класса-предка быть не может.
- {Incomplete} - означает случай, противоположный первому. А именно, предполагается, что на диаграмме указаны не все классы-потомки. В последующем, возможно, разработчик восполнит их перечень, не изменяя уже построенную диаграмму.
- {Disjoint} - означает, что классы-потомки не могут содержать объектов, одновременно являющихся экземплярами двух или более классов.
- {Overlapping} - случай, противоположный предыдущему. А именно, предполагается, что отдельные экземпляры классов-потомков могут принадлежать одновременно нескольким классам.

С учетом дополнительного использования стандартного ограничения диаграмма классов (рисунок 19) может быть уточнена (рисунок 20).



Рисунок 20 - Вариант уточненного графического изображения отношения обобщения классов с использованием строки-ограничения

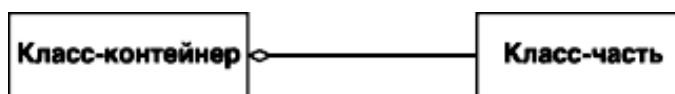


Рисунок 21 - Графическое изображение отношения агрегации в языке UML

Отношение агрегации имеет место между несколькими классами в том случае, если один из классов представляет собой сущность, которая включает в себя в качестве составных частей другие сущности. Данное отношение применяется для представления системных взаимосвязей типа «часть-целое» и показывает, из каких элементов состоит система, и как они связаны между собой.

Очевидно, что рассматриваемое в таком аспекте деление системы на составные части представляет собой иерархию, но принципиально отличную от той, которая порождается отношением обобщения. Отличие заключается в том, что части системы никак не обязаны наследовать ее свойства и поведение, поскольку являются самостоятельными сущностями. Более того, части целого обладают собственными атрибутами и операциями, которые существенно отличаются от атрибутов и операций целого.

Графически отношение агрегации изображается сплошной линией, один из концов которой представляет собой не закрашенный внутри ромб. Этот ромб указывает на тот класс, который представляет собой «целое» или класс-контейнер. Остальные классы являются его «частями» (рисунок 22).

В качестве примера отношения агрегации можно рассмотреть взаимосвязь типа «часть-целое», которая имеет место между классом «Системный блок» персонального компьютера и его составными частями: «Процессор», «Материнская плата», «Оперативная память», «Жесткий диск» и «Дисковод гибких дисков».



Рисунок 22 - Диаграмма классов для иллюстрации отношения агрегации на примере системного блока ПК

Отношение композиции - частный случай отношения агрегации, и служит для спецификации более сильной формы отношения «часть-целое», при которой составляющие части тесно взаимосвязаны с целым. Особенность: с уничтожением целого уничтожаются и все его составные части.

Композит (composite) - класс, который связан отношением композиции с одним или большим числом классов. Графически отношение композиции изображается сплошной линией, один из концов которой представляет собой закрашенный внутри ромб. Этот ромб указывает на тот класс, который представляет собой класс-композит. Остальные классы являются его «частями» (рисунок 23).



Рисунок 23 - Графическое изображение отношения агрегации «Класс-композит» и «Класс-часть»

Для отношений композиции и агрегации могут использоваться дополнительные обозначения, применяемые для отношения ассоциации. А именно, могут указываться кратности отдельных классов, которые в общем случае не обязательны. Применительно к описанному выше примеру класс Окно программы является классом-композицией, а взаимосвязи составляющих его частей могут быть изображены следующей диаграммой классов (рисунок 24).



Рисунок 24 - Диаграмма классов для иллюстрации отношения композиции на примере класса-композиции «Окно программы»

От умения правильно выбрать классы и установить между ними взаимосвязи часто зависит не только успех процесса проектирования, но и производительность выполнения программы.

При определении классов, атрибутов и операций, задании их имен и типов перед разработчиком всегда встает вопрос: какой язык взять за основу.

Пример (рисунок 25). Информационная система содержит базу данных «Автоматизация процесса составления расписания в учебном заведении».

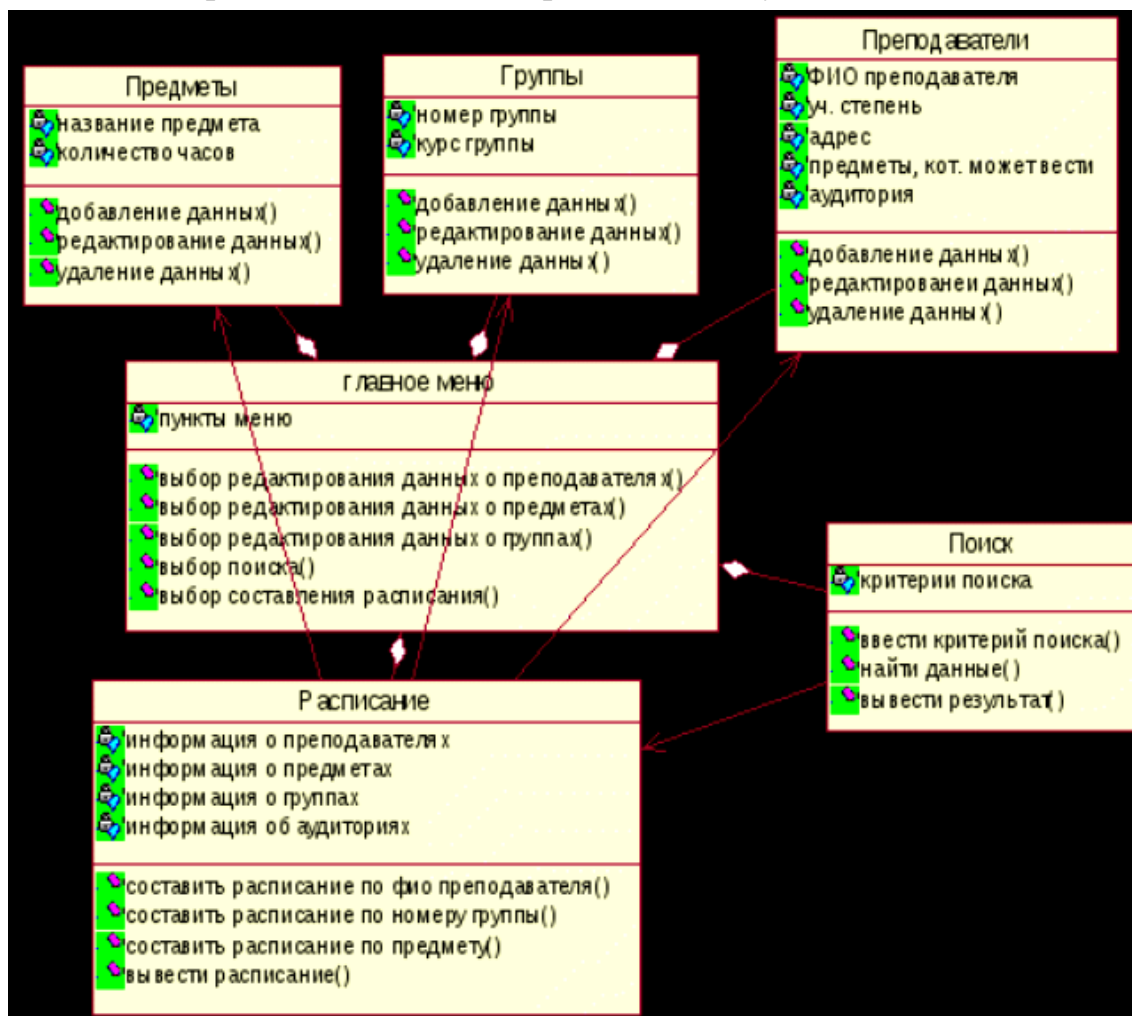


Рисунок 25 - Пример диаграммы классов

Система обеспечивает корректировку данных, а именно в БД «Группы» может изменяться перечень предметов в соответствии с курсом группы и отделением; в БД «Предметы» могут изменяться номера аудиторий и фамилии преподавателей; осуществляет поиск по ФИО преподавателя, номеру аудитории, названию предмета и номеру группы.

Система составляет расписание работы: для конкретного преподавателя на

неделю; для группы на неделю; для группы по конкретному предмету, а также отчет о загрузке аудиторий на каждый день.

4.4. Диаграмма деятельности

Диаграмма деятельности (Activity diagram) — диаграмма, на которой показано разложение некоторой деятельности на её составные части.

Под деятельностью (англ. activity) понимается спецификация исполняемого поведения в виде координированного последовательного и параллельного выполнения подчинённых элементов — вложенных видов деятельности и отдельных действий (англ. action), соединённых между собой потоками, которые идут от выходов одного узла к входам другого.

Диаграммы деятельности используются при моделировании бизнес-процессов, технологических процессов, последовательных и параллельных вычислений.

Аналогом диаграмм деятельности являются схемы алгоритмов по ГОСТ 19.701–90.

При моделировании поведения проектируемой или анализируемой программной системы возникает необходимость не только представить процесс изменения ее состояний, но и детализировать особенности алгоритмической и процедурной реализации выполняемых системой операций. Для этой цели, как правило, используются блок-схемы или структурные схемы алгоритмов.

Каждая такая схема акцентирует внимание на последовательности выполнения определенных процедур или элементарных операций, которые в совокупности приводят к получению желаемого результата. С увеличением сложности системы строгое соблюдение определенной последовательности выполняемых действий приобретает большое значение.

Для моделирования процесса выполнения операций в языке UML используются диаграммы деятельности. Применяемая в них графическая нотация во многом похожа на нотацию диаграммы состояний, поскольку на диаграммах деятельности также присутствуют обозначения состояний и переходов. Отличие заключается в семантике состояний, которые используются для представления деятельности и действий, а также в отсутствии на переходах сигнатуры событий.

Каждое состояние на диаграмме деятельности соответствует выполнению некой операции, а переход в следующее состояние происходит только после завершения выполнения этой операции. Диаграмма деятельности представляется

в форме графа деятельности, вершинами которого являются состояния действия или деятельности, а дугами - переходы от одного состояния действия к другому.

Диаграммы деятельности (Activity diagram)- частный случай диаграмм состояний. Они позволяют реализовать в языке UML особенности процедурного и синхронного управления, обусловленного завершением внутренних действий и деятельности. Основным направлением использования диаграмм деятельности является визуализация особенностей реализации операций классов, когда необходимо представить алгоритмы их выполнения. При этом каждое состояние может являться выполнением операции определенного класса либо ее части, позволяя использовать диаграммы деятельности для описания реакций на внутренние события системы.

В контексте языка UML *деятельность* представляет собой совокупность отдельных вычислений, выполняемых автоматом. При этом отдельные элементарные вычисления могут приводить к результату или действию.

На диаграмме деятельности отображается логика или последовательность перехода от одной деятельности к другой, при этом внимание фиксируется на результате деятельности. Сам же результат может привести к изменению состояния системы или возвращению некоторого значения.

Диаграмма деятельности предназначена для моделирования поведения систем, хотя время в явном виде отсутствует на этой диаграмме.

Состояние деятельности (Activity state) - состояние в графе деятельности, которое служит для представления процедурной последовательности действий, требующих определенного времени.

Переход из состояния деятельности происходит после выполнения специфицированной в нем деятельности, при этом ключевое слово *do* в имени деятельности не указывается. Состояние деятельности не может иметь внутренних переходов, поскольку оно является элементарным.

Состояния деятельности могут быть подвергнуты дальнейшей декомпозиции, вследствие чего выполняемую деятельность можно представить с помощью других диаграмм деятельности.

Состояния деятельности не являются атомарными, то есть могут быть прерваны. Предполагается, что для их завершения требуется заметное время.

Состояние деятельности можно представлять себе как составное состояние, поток управления которого включает только другие состояния деятельности и действий (рисунок 26).

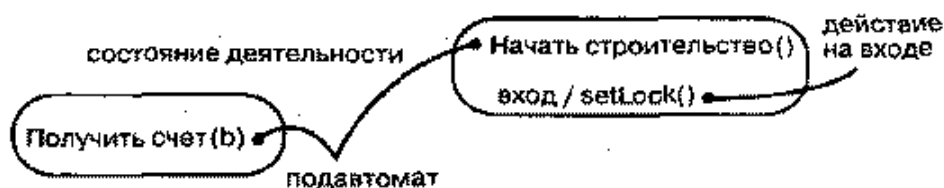


Рисунок 26 - Графическое изображение состояний деятельности действия

Состояние действия (Action state) - специальный случай состояния с некоторым входным действием и, по крайней мере, одним выходящим из состояния переходом. Переход из состояния действия происходит после завершения входного действия.

Состояние действия не может иметь внутренних переходов, поскольку оно является элементарным. Использование состояния действия заключается в моделировании шага выполнения алгоритма.

Графически состояния деятельности и действия изображаются одинаковой фигурой, напоминающей прямоугольник, боковые стороны которого заменены выпуклыми дугами. Внутри этой фигуры записывается имя состояния деятельности или действия в форме выражения, которое должно быть уникальным в пределах одной диаграммы деятельности.

Действие может быть записано на естественном языке, псевдокоде или языке программирования. Никаких дополнительных или неявных ограничений при записи действий не накладывается. Рекомендуется в качестве имени простого действия использовать глагол с пояснительными словами (рисунок 27). Если же действие может быть представлено в формальном виде, то оно записывается на том языке программирования, на котором предполагается реализовывать разрабатываемый проект.

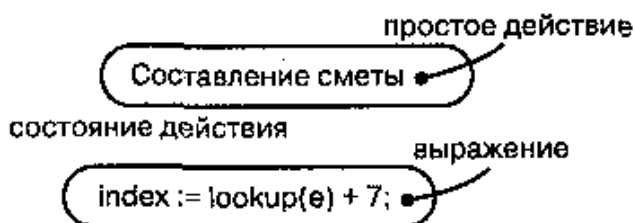


Рисунок 27 - Графическое изображение состояний действия на диаграмме деятельности

Когда возникает необходимость представить на диаграмме деятельности сложное действие, состоящее из нескольких более простых, то используют специальное обозначение так называемого состояния под-деятельности.

Состояние под-деятельности (Subactivity state) - состояние в графе деятельности, которое служит для представления неатомарной последовательности шагов процесса. Это состояние является графом деятельности и обозначается специальной пиктограммой в правом нижнем углу символа состояния действия (рисунок 28).

Данная конструкция может применяться к любому элементу языка UML, который поддерживает «вложенность» своей структуры. При этом пиктограмма может быть дополнительно помечена типом вложенной структуры.

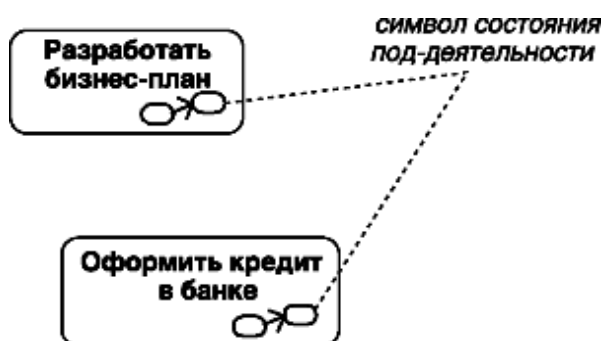


Рисунок 28 - Графическое изображение состояния под-деятельности

Каждая диаграмма деятельности должна иметь единственное начальное и конечное состояния. Они имеют такие же обозначения, как и на диаграмме состояний. При этом каждая деятельность начинается в начальном состоянии и заканчивается в конечном состоянии.

Саму диаграмму деятельности принято располагать таким образом, чтобы действия следовали сверху вниз или слева направо. В этом случае начальное состояние будет изображаться в верхней или левой части диаграммы, а конечное - в ее нижней или правой части. В интересах удобства визуального представления на диаграмме деятельности допускается изображать несколько конечных состояний. В этом случае все их принято считать эквивалентными друг другу.

Переход на диаграмме деятельности аналогичен переходу на диаграмме состояний. При построении диаграммы деятельности используются только

нетриггерные переходы, т. е. такие, которые происходят сразу после завершения деятельности или выполнения соответствующего действия. Такой переход передает управление в последующее состояние сразу, как только закончится действие или деятельность в предыдущем состоянии. На диаграмме такой переход изображается сплошной линией со стрелкой.

Если из состояния действия выходит единственный переход, то его можно никак не помечать. Если же таких переходов несколько, то при моделировании последовательной деятельности запускается только один из них. В этом случае для каждого из таких переходов должно быть явно записано собственное сторожевое условие в прямых скобках.

При этом для всех выходящих из некоторого состояния деятельности переходов должно выполняться требование истинности только одного из них. Подобный случай встречается тогда, когда последовательно выполняемая деятельность должна разделиться на альтернативные ветви в зависимости от значения промежуточного результата. Такая ситуация получила название ветвления, а для ее обозначения применяется специальный символ решения.

Графически ветвление на диаграмме деятельности обозначается символом решения (decision), изображаемого в форме небольшого ромба, внутри которого нет никакого текста (рисунок 29).

В ромб может входить только одна стрелка от того состояния действия, после выполнения, которого поток управления должен быть продолжен по одной из взаимно исключающих ветвей. Принято входящую стрелку присоединять к верхней или левой вершине символа решения. Выходящих стрелок может быть две или более. Для каждой из них явно указывается соответствующее сторожевое условие в форме булевского выражения.

Для графического объединения альтернативных ветвей на диаграмме деятельности рекомендуется использовать аналогичный символ в форме ромба, который в этом случае называют соединением (merge).

Наличие этого символа, внутри которого также не записывается никакого текста, упрощает визуальный контроль логики выполнения процедурных действий на диаграмме деятельности (рисунок 29 внизу). Входящих стрелок у символа соединения может быть несколько, они исходят от состояний действия, принадлежащих к одной из взаимно исключающих ветвей. Выходить из ромба соединения может только одна стрелка, при этом ни входящие, ни выходящая стрелки не должны содержать сторожевых условий. Исключением является

ситуация, когда с целью сокращения диаграммы объединяют символ решения с символом соединения.

Пример (рисунок 29). Моделируется ситуация, возникающая в супермаркетах при оплате товаров. Как правило, заплатить за покупки можно либо наличными, либо по кредитной карточке. Если покупателем выбран вариант оплаты по кредитной карточке, то проверяется сумма баланса предъявленной к оплате кредитной карточки. При этом оплата происходит только в том случае, если общая стоимость приобретаемых товаров не превышает суммы баланса этой карточки. В противном случае оплаты не происходит, и товар остается у продавца.

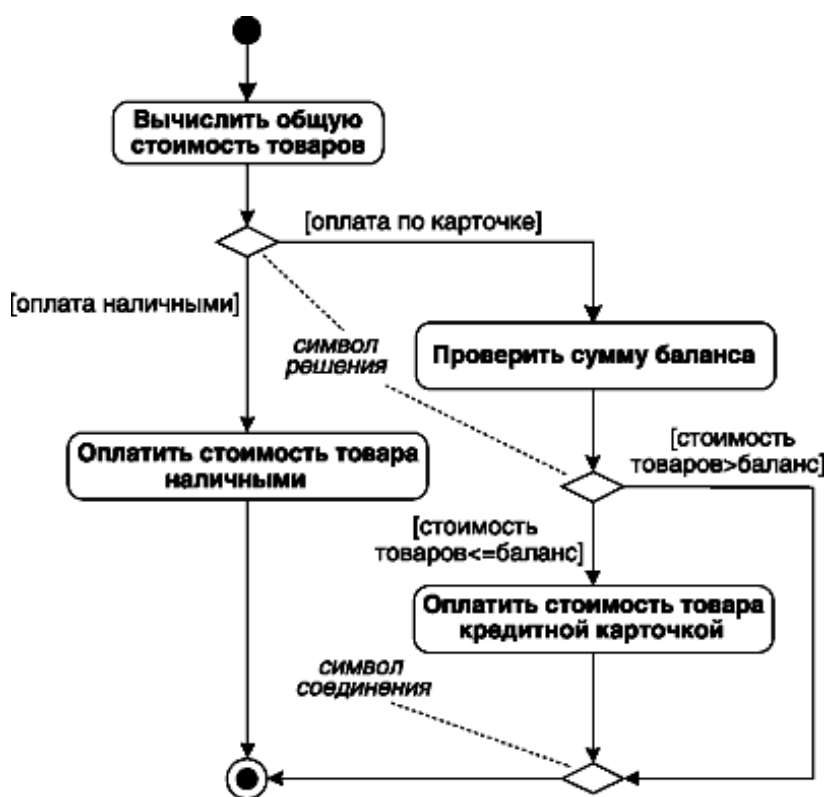


Рисунок 29 - Различные варианты ветвлений на диаграмме деятельности

На диаграмме деятельности изображаются параллельные процессы, поскольку распараллеливание вычислений существенно повышает быстродействие программных систем. В диаграммах деятельности для изображения параллельных процессов используется специальный символ.

На диаграммах деятельности такая черточка изображается отрезком горизонтальной, реже - вертикальной, линии, толщина которой несколько шире линий простых переходов диаграммы деятельности. При этом разделение (fork)

имеет один входящий переход и несколько выходящих (рисунок 30, а), которые изображаются отрезками вертикальных, реже - горизонтальных, линий.

Слияние (join), наоборот, имеет несколько входящих переходов и один выходящий (рисунок 30, б). Параллельные переходы на диаграмме деятельности можно изображать в удлинненной форме, а входящие и выходящие переходы вертикальными стрелками.

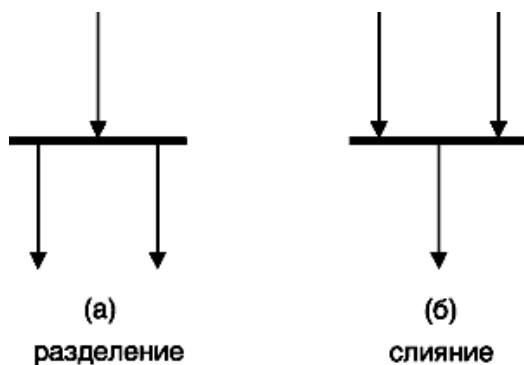


Рисунок 30 - Графическое изображение разделения и слияния параллельных потоков управления на диаграмме деятельности

Пример. Регистрация пассажиров в аэропорту (рисунок 31).

Первоначально выполняется деятельность по проверке билета: если билет не действителен, он возвращается пассажиру; если билет действителен, то пассажиру выдается посадочный талон, в дополнение проверяется гражданство и наличие багажа у пассажира.

Если есть багаж, то его проверка может быть выполнена параллельно, после чего пассажиру выдается талон на багаж.

Если пассажир является иностранным гражданином, то дополнительно проверяется наличие у него визы. Если виза действительна, то проверка завершается успешно, и пассажир может проследовать на посадку.

Если же виза не действительна, то для этого пассажира посадка оказывается невозможной, и ему не выдается посадочный талон и талон на багаж. Происходит прекращение всех выполняемых сотрудниками аэропорта действий.

Диаграммы деятельности могут быть использованы не только для спецификации алгоритмов вычислений или потоков управления в программных системах. Не менее важная область их применения связана с моделированием бизнес-процессов. В этом контексте деятельность любой компании или фирмы

представляет собой не что иное, как совокупность отдельных действий, работ, операций, направленных на достижение требуемого результата.

Однако применительно к бизнес-процессам желательно выполнение каждого действия ассоциировать с конкретным подразделением компании. В этом случае подразделение будет нести ответственность за реализацию определенных действий, а сам бизнес-процесс представляется в виде переходов действий из одного подразделения к другому. Для моделирования этих особенностей в языке UML предложена специальная конструкция, получившая название дорожки.

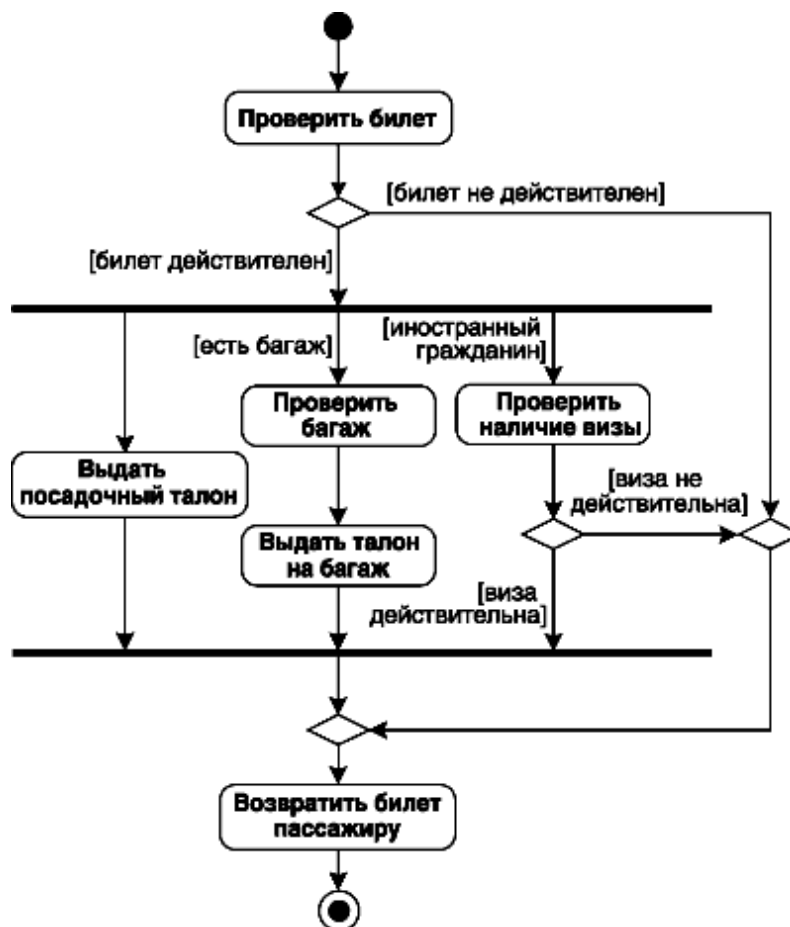


Рисунок 31 - Диаграмма деятельности для примера регистрации пассажиров в аэропорту

Дорожка (swimlane) - графическая область диаграммы деятельности, содержащая элементы модели, ответственность за выполнение которых принадлежит отдельным подсистемам.

В данном случае имеется в виду визуальная аналогия с плавательными дорожками в бассейне, если смотреть на соответствующую диаграмму

деятельности сверху. При этом все состояния на диаграмме деятельности делятся на группы, разграниченные вертикальными линиями.

Две соседних линии и образуют дорожку, а группа состояний между этими линиями выполняется организационным подразделением (отделом, группой, отделением, филиалом) или сотрудником компании (рисунок 32). В последнем случае принято указывать должность сотрудника, ответственного за выполнение определенных действий.

Названия подразделений или должностей явно указываются в верхней части дорожки. Пересекать линию дорожки могут только переходы, которые в этом случае обозначают выход или вход потока управления в соответствующее подразделение компании. Порядок следования дорожек не несет какой-либо семантической информации и определяется соображениями удобства.

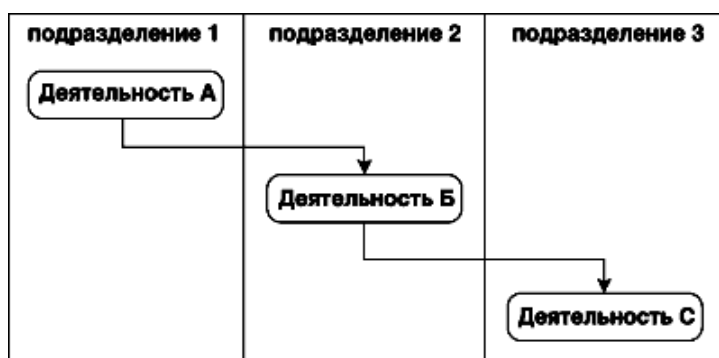


Рисунок 32 - Вариант диаграммы деятельности с дорожками

Пример (рисунок 33) - фрагмент диаграммы деятельности торговой компании, обслуживающей клиентов в форме заказов.

Подразделениями компании обычно являются отдел приема и оформления заказов, отдел продаж и склад. Этим подразделениям будут соответствовать три дорожки на диаграмме деятельности, каждая из которых специфицирует зону ответственности подразделения. В этом случае диаграмма деятельности включает в себе не только информацию о последовательности выполнения рабочих действий, но и о том, какое подразделение торговой компании должно выполнять то или иное действие.

После принятия заказа от клиента отделом приема и оформления заказов осуществляется распараллеливание деятельности на два потока (переход-разделение).

Первый из них остается в этом же отделе и связан с получением оплаты от клиента за заказанный товар.

Второй инициирует выполнение действия по регистрации заказа в отделе продаж (модель товара, размеры, цвет, год выпуска и пр.). Однако выдача товара со склада начинается только после того, как будет получена от клиента оплата за товар (переход-слияние).

Затем выполняется подготовка товара к отправке и его отправка клиенту в отделе продаж. После завершения этих деятельности заказ закрывается в отделе приема и оформления заказов.

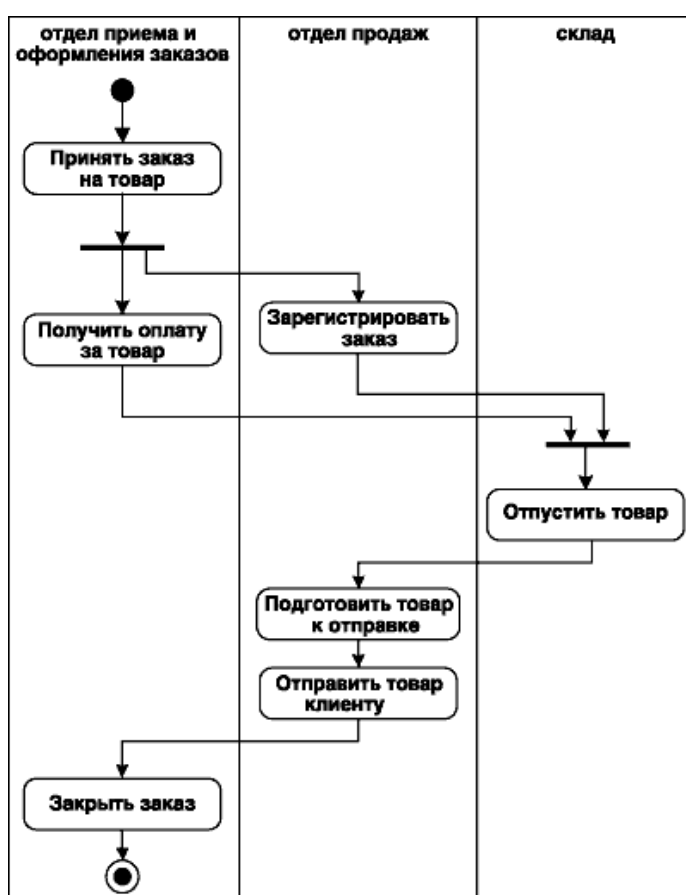


Рисунок 33 - Фрагмент диаграммы деятельности для торговой компании

Действия на диаграмме деятельности могут производиться над теми или иными объектами. Эти объекты либо инициируют выполнение действий, либо определяют результат этих действий. При этом действия специфицируют вызовы, которые передаются от одного объекта графа деятельности другому. Поскольку в таком ракурсе объекты играют определенную роль в понимании

процесса деятельности, иногда возникает необходимость явно указать их на диаграмме деятельности.

На диаграмме деятельности с дорожками расположение объекта может иметь дополнительный смысл. А именно, если объект расположен на границе двух дорожек, то это может означать, что переход к следующему состоянию действия в соседней дорожке ассоциирован с нахождением документа в некотором состоянии. Если же объект расположен внутри дорожки, то и состояние этого объекта целиком определяется действиями данной дорожки.

В примере с торговой компанией центральным объектом процесса продажи является заказ или вернее состояние его выполнения. Вначале, до обращения клиента, заказ как объект отсутствует и возникает после контакта с клиентом.

В результате фиксируется полученный заказ, потом он регистрируется в отделе продаж, затем он передается на склад, где после получения оплаты за товар оформляется окончательно. После того, как товар отправлен клиенту, эта информация вносится в заказ, и он считается выполненным (рисунок 34).

Достоинством диаграммы деятельности является возможность визуализировать отдельные аспекты поведения рассматриваемой системы или реализации отдельных операций классов в виде процедурной последовательности действий.

Таким образом, полная модель системы может содержать одну или несколько диаграмм деятельности, каждая из которых описывает последовательность реализации либо наиболее важных вариантов использования (типичный ход событий и все исключения), либо нетривиальных операций классов.

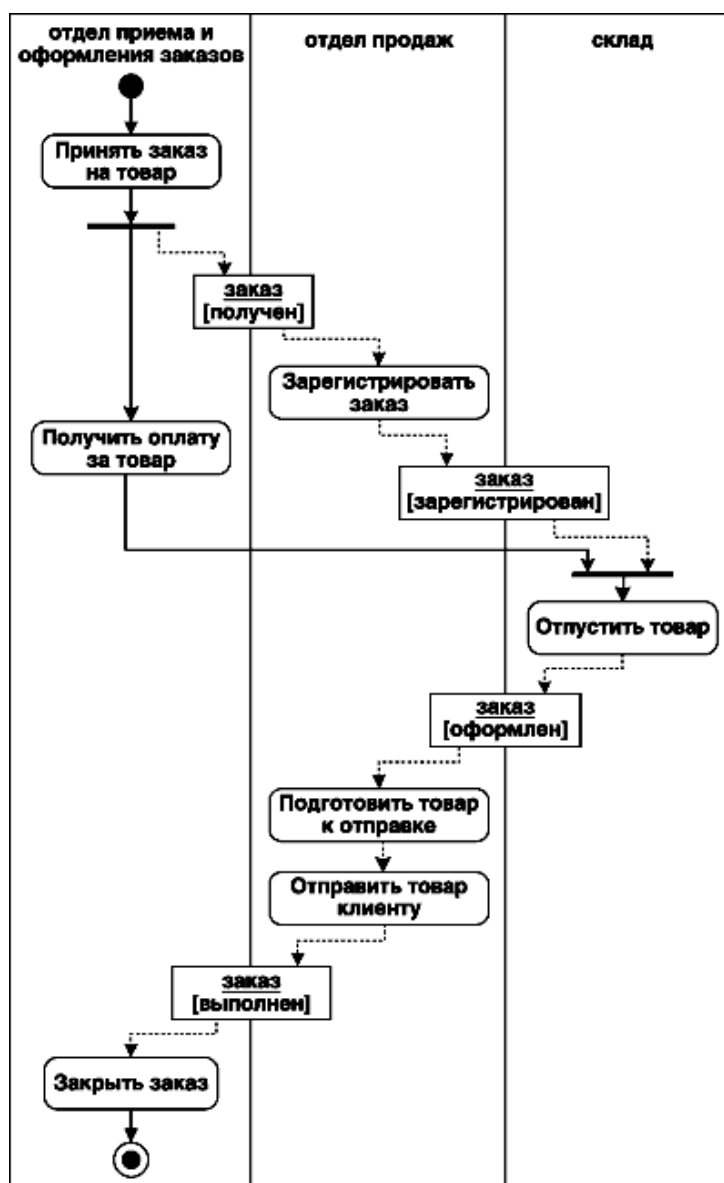


Рисунок 34 - Фрагмент диаграммы деятельности торговой компании с объектом-заказом

Пример (рисунок 35). Информационная система представляет собой базу данных «Автоматизация процесса составления расписания в учебном заведении».

Система обеспечивает корректировку данных, а именно в БД «Группы» может изменяться перечень предметов в соответствие с курсом группы и отделением; в БД «Предметы» могут изменяться номера аудиторий и фамилии преподавателей; осуществляет поиск по ФИО преподавателя, номеру аудитории, названию предмета и номеру группы.

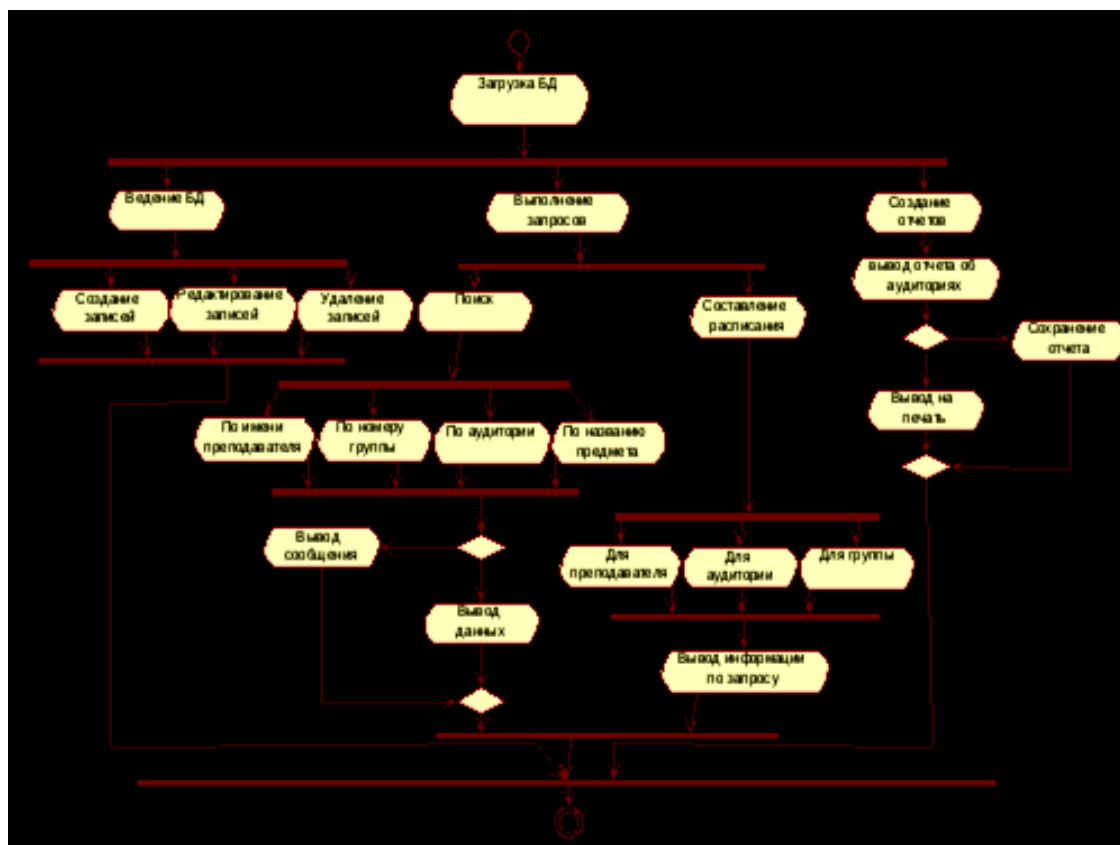


Рисунок 35 - Пример диаграммы деятельности

Система составляет расписание работы для конкретного преподавателя на неделю; для группы на неделю; для группы по конкретному предмету, а также отчет о загрузке аудиторий на каждый день.

Литература

1. ГОСТ Р ИСО 15704–2008. Требования к стандартным архитектурам и методологиям предприятия. - М.: Стандартинформ, 2010.
2. Глод, О.Д. Архитектура предприятия: учебное пособие / О.Д. Глод; Южный федеральный университет. - Таганрог: Издательство Южного федерального университета, 2016. - 93 с.
3. Гриценко, Ю. Б. Архитектура предприятия [Электронный ресурс] : учеб. пособие / Гриценко Ю. Б. – Томск: Томский государственный университет систем управления и радиоэлектроники, 2011. – 264 с. – Режим доступа : <http://www.iprbookshop.ru/14005.html>
4. Данилин, А. Архитектура предприятия [Электронный ресурс] / А. Данилин, А. Слюсаренко. — Электрон. текстовые данные. — М. : Интернет-Университет Информационных Технологий (ИНТУИТ), 2016. — 439 с. – Режим доступа : <http://www.iprbookshop.ru/62807.html>
5. Желваков, Б. Б. Архитектура корпоративных информационных систем : учеб. пособие / Б. Б. Желваков. – СПб.: Санкт-Петербургский гос. инж.-эконом ун-т, 2012. – 622 с.
6. Калянов, Г.Н. Моделирование, анализ, реорганизация и автоматизация бизнес-процессов : учеб. пособие для вузов / Г.Н. Калянов. – М. : Финансы и статистика, 2006. – 240 с.
7. Крэг Ларман. Применение UML 2.0 и шаблонов проектирования = Applying UML and Patterns : An Introduction to Object-Oriented Analysis and Design and Iterative Development. — 3-е изд. — М.: Вильямс, 2006. — 736 с. — ISBN 0-13-148906-2
8. Буч Г., Якобсон А., Рамбо Дж. UML. Классика CS. 2-е изд. / Пер. с англ.; под общей редакцией проф. С. Орлова — СПб.: Питер, 2006. — 736 с.
9. Буч Г. Объектно-ориентированный анализ и проектирование с примерами приложений, 3-е изд. / Буч Градди, Максимчук Роберт А., Энгл Майкл У., Янг Бобби Дж., Коналлен Джим, Хьюстон Келли А.: Пер с англ. — М.: ООО «И. Д. Вильямс», 2010. — 720 с.
10. Леоненков, А.В. Самоучитель UML 2 / А. В. Леоненков. – СПб.: БХВ — Петербург, 2007. – 576 с.
11. Арлоу Дж., Нейштадт А. UML 2 и Унифицированный процесс. Практический объектно-ориентированный анализ и проектирование. – Символ-Плюс, 2007. – 624 с.

Оглавление

1. Программа работы	2
ЗАДАНИЕ 1. РАЗРАБОТКА ГЛОССАРИЯ.....	2
УКАЗАНИЯ К ВЫПОЛНЕНИЮ ЗАДАНИЯ 1	2
ЗАДАНИЕ 2. РАЗРАБОТКА ДИАГРАММЫ ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ	
(ПРЕЦЕДЕНТОВ) <i>UML</i> ДЛЯ ЗАДАННОЙ СИСТЕМЫ	3
УКАЗАНИЯ К ВЫПОЛНЕНИЮ ЗАДАНИЯ 2.....	3
ЗАДАНИЕ 3. РАЗРАБОТКА ДИАГРАММЫ КЛАССОВ <i>UML</i> ДЛЯ ЗАДАННОЙ	
СИСТЕМЫ.....	6
ЗАДАНИЕ 4. РАЗРАБОТКА ДИАГРАММЫ ДЕЯТЕЛЬНОСТИ <i>UML</i> ДЛЯ ЗАДАННОЙ	
СИСТЕМЫ.....	7
2. Исходные данные	7
3. Отчетность	8
4. Методические указания	9
4.1. УНИФИЦИРОВАННЫЙ ЯЗЫК МОДЕЛИРОВАНИЯ <i>UML</i> БИЗНЕС-	
ПРОЦЕССОВ И СИСТЕМ.....	9
4.2. ДИАГРАММА ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ	11
4.3. ДИАГРАММА КЛАССОВ	22
4.4. ДИАГРАММА ДЕЯТЕЛЬНОСТИ	39
Литература.....	52